

Práctica: Contornos y Aplicaciones

Objetivo

Aprender a detectar, analizar y comparar contornos con OpenCV, y aplicar estos conceptos a problemas reales como conteo de objetos y videovigilancia.

Introducción a los contornos

Un contorno es la línea que rodea un objeto dentro de una imagen binaria.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('objetos.jpg')
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binaria = cv2.threshold(gris, 120, 255, cv2.THRESH_BINARY)

contornos, jerarquia = cv2.findContours(
    binaria,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

resultado = img.copy()
cv2.drawContours(resultado, contornos, -1, (0,255,0), 2)

plt.figure(figsize=(10,5))
plt.imshow(cv2.cvtColor(resultado, cv2.COLOR_BGR2RGB))
plt.title('Contornos detectados')
plt.axis('off')
plt.show()

print("Cantidad de contornos:", len(contornos))
```

Actividad: Probar con una imagen de objetos simples y cambiar el umbral y observar cómo cambia el número de contornos.

Conteo de monedas mediante contornos

```
img = cv2.imread('monedas.jpg')
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gris, (5,5), 0)

_, binaria = cv2.threshold(blur, 100, 255, cv2.THRESH_BINARY_INV)

contornos, _ = cv2.findContours(
    binaria,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

```

resultado = img.copy()

for c in contornos:
    cv2.drawContours(resultado, [c], -1, (0,255,0), 2)

plt.figure(figsize=(10,5))
plt.imshow(cv2.cvtColor(resultado, cv2.COLOR_BGR2RGB))
plt.title(f'Monedas detectadas: {len(contornos)}')
plt.axis('off')
plt.show()

```

Actividad: Contar cuántas monedas aparecen y verificar si una moneda parcialmente oculta también se detecta.

Conteo de tornillos y discriminación por tamaño

En esta práctica se eliminan contornos muy pequeños usando el área o un kernel morfológico.

```

img = cv2.imread('tornillos.jpg')
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binaria = cv2.threshold(gris, 120, 255, cv2.THRESH_BINARY_INV)

kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (7,7))
binaria = cv2.morphologyEx(binaria, cv2.MORPH_OPEN, kernel)

contornos, _ = cv2.findContours(binaria, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

resultado = img.copy()

contador = 0

for c in contornos:
    area = cv2.contourArea(c)

    if area > 500:
        contador += 1
        cv2.drawContours(resultado, [c], -1, (255,0,0), 2)

plt.figure(figsize=(10,5))
plt.imshow(cv2.cvtColor(resultado, cv2.COLOR_BGR2RGB))
plt.title(f'Tornillos detectados: {contador}')
plt.axis('off')
plt.show()

```

Actividad: Cambiar el tamaño del kernel (3,3), (7,7) y (15,15) y explicar qué objetos desaparecen.

Centroide de un contorno

El centroide es el “centro de masa” de un objeto.

```

resultado = img.copy()

```

```

for c in contornos:
    M = cv2.moments(c)

    if M["m00"] != 0:
        cx = int(M["m10"] / M["m00"])
        cy = int(M["m01"] / M["m00"])

        cv2.circle(resultado, (cx, cy), 5, (0,0,255), -1)
        cv2.putText(resultado, f'({cx},{cy})', (cx+10, cy),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,0,0), 1)

plt.imshow(cv2.cvtColor(resultado, cv2.COLOR_BGR2RGB))
plt.title('Centroide de cada contorno')
plt.axis('off')
plt.show()

```

Actividad:

- Marcar el centro de cada moneda o tornillo.
- Determinar cuál está más cerca del centro de la imagen.

Discriminación de contornos mediante área

```

resultado = img.copy()

for c in contornos:
    area = cv2.contourArea(c)

    if area > 1000:
        color = (0,255,0)
    else:
        color = (255,0,0)

    cv2.drawContours(resultado, [c], -1, color, 2)

plt.imshow(cv2.cvtColor(resultado, cv2.COLOR_BGR2RGB))
plt.title('Contornos clasificados por área')
plt.axis('off')
plt.show()

```

Interpretación:

- Verde = objeto grande
- Azul = objeto pequeño

Actividad: Separar monedas grandes y pequeñas y clasificar tornillos y tuercas.

Prueba de polígono de puntos

Permite saber si un punto está dentro o fuera de un contorno.

```

punto = (150, 200)

for c in contornos:
    resultado = cv2.pointPolygonTest(c, punto, False)

```

```
if resultado >= 0:
    print("El punto está dentro del contorno")
else:
    print("El punto está fuera")
```

Actividad: Elegir varios puntos y identificar si están dentro de una moneda o fuera.

Comparación de contornos de dos engranajes

Se puede medir si dos objetos tienen forma similar.

```
img1 = cv2.imread('engranaje1.png', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('engranaje2.png', cv2.IMREAD_GRAYSCALE)

_, th1 = cv2.threshold(img1, 127, 255, cv2.THRESH_BINARY)
_, th2 = cv2.threshold(img2, 127, 255, cv2.THRESH_BINARY)

c1, _ = cv2.findContours(th1, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
c2, _ = cv2.findContours(th2, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

valor = cv2.matchShapes(c1[0], c2[0], 1, 0)

print("Diferencia entre contornos:", valor)
```

Interpretación:

- Valor cercano a 0 : muy parecidos
- Valor alto : diferentes

Actividad: Comparar dos objetos iguales y dos diferentes y explicar cuándo sería útil esta técnica en industria.

Sistema de videovigilancia con detección de personas y zona segura

Se puede definir una región segura y detectar si una persona ingresa.

```
video = cv2.VideoCapture('personas.mp4')

hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

ret, frame = video.read()

zona = [(100,100), (500,100), (500,400), (100,400)]

while ret:
    boxes, weights = hog.detectMultiScale(frame, winStride=(8,8))

    for (x,y,w,h) in boxes:
        cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)

        centro = (x + w//2, y + h//2)
```

```
        if 100 < centro[0] < 500 and 100 < centro[1] < 400:
            cv2.putText(frame, 'ALERTA', (x,y-10),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

    cv2.rectangle(frame, zona[0], zona[2], (255,0,0), 2)

    plt.figure(figsize=(10,6))
    plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    plt.axis('off')
    plt.show()

    break
```

Actividad:

- Definir una nueva zona segura.
- Probar con un video de personas caminando.
- Generar una alerta si alguien entra.